Software Engineering Questions - 3 Mark Answers

1. What is Software engineering?

Software engineering is a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software. It applies engineering principles to software creation, aiming to produce reliable, efficient, and high-quality software products within budget and on schedule. It encompasses various methodologies, tools, and procedures to manage the entire software lifecycle.

2. What is software? List characteristics.

Software refers to a set of instructions, data, or programs used to operate computers and execute specific tasks. It is intangible and does not wear out in the traditional sense, but it can deteriorate due to changes in requirements or environment.

Characteristics of Software:

- **Intangibility:** Software is not a physical element; it's logical.
- **Complexity:** Software can be highly complex, involving millions of lines of code.
- **Changeability:** Software is constantly evolving to meet new requirements.
- **Non-wearing out:** Unlike hardware, software does not physically wear out but can become obsolete.
- **Manufactured, not assembled:** Software is developed or engineered, not assembled from components.

3. What is software process?

A software process (also known as a software development life cycle - SDLC) is a structured set of activities required to develop a software system. These activities typically include specification, design, implementation, validation, and evolution. The goal of a software process is to provide a framework for building high-quality software efficiently and effectively, ensuring that all necessary steps are followed in a systematic manner.

4. What are the phases of software development lifecycle?

The Software Development Life Cycle (SDLC) is a framework defining tasks performed at each step in the software development process. The typical phases include:

- **Requirement Gathering and Analysis:** Understanding and documenting user needs.
- **Design:** Planning the architecture and components of the software.
- **Implementation/Coding:** Writing the actual code based on the design.
- **Testing:** Verifying that the software meets requirements and is free of defects.
- **Deployment:** Releasing the software to the users.
- **Maintenance:** Ongoing support, updates, and enhancements after deployment.

5. Define Evolutionary process model.

Evolutionary process models are iterative software development approaches that allow for the gradual refinement of software through successive versions. Instead of delivering a complete system at once, these models develop an initial version, gather feedback, and then evolve the software in subsequent iterations. This approach is particularly useful when requirements are unclear or expected to change, as it allows for flexibility and continuous improvement. Examples include prototyping and spiral models.

6. State benefits and drawbacks of waterfall cycle model.

The Waterfall model is a linear, sequential approach to software development.

Benefits:

- **Simplicity:** Easy to understand and manage due to its rigid structure.
- **Clear Phases:** Well-defined stages with specific deliverables and review points.
- **Early Error Detection (in theory):** Errors are ideally caught in earlier phases, though this is often not the case in practice.

Drawbacks:

- **Lack of Flexibility:** Difficult to accommodate changes once a phase is completed.
- **Late Testing:** Testing occurs late in the cycle, making it costly to fix errors.
- **No Early Customer Feedback:** Customers only see the product at the very end, leading to potential dissatisfaction.
- **High Risk and Uncertainty:** Not suitable for complex or long-term projects with evolving requirements.

7. Write the IEEE definition of software engineering, categories of software.

IEEE Definition of Software Engineering:

> "The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software."

Categories of Software:

- **System Software:** Operating systems, device drivers, utilities.
- **Application Software:** Word processors, spreadsheets, web browsers.
- **Embedded Software:** Software in consumer products (e.g., microwaves, cars).
- **Web/Mobile Applications:** Software accessed via browsers or mobile devices.
- **Artificial Intelligence Software:** AI/ML applications, expert systems.
- **Scientific/Engineering Software:** CAD, simulation software.

8. Define incremental process model.

The Incremental process model combines elements of the Waterfall model with iterative development. In this model, the software is broken down into multiple standalone modules or increments, each developed through a mini-Waterfall cycle (requirements, design, code, test). Each increment adds functionality to the previous one, with the final product being the sum of all increments. This allows for early delivery of operational software and provides opportunities for customer feedback at each stage.

9. What are the goals and objectives of software?

The primary goals and objectives of software are to solve real-world problems, automate tasks, and enhance human capabilities.

Key Objectives:

- **Functionality:** To perform specific tasks and meet user requirements.
- **Reliability:** To operate correctly and consistently without failure.
- **Efficiency:** To utilize resources (time, memory, CPU) effectively.
- **Usability:** To be easy to learn, operate, and understand by its users.
- **Maintainability:** To be easily modifiable, adaptable, and fixable over time.
- **Portability:** To be able to run on different platforms with minimal changes.
- **Security:** To protect data and system from unauthorized access or damage.

10. Define RAD model.

RAD stands for **Rapid Application Development**. It is an incremental software development process model that emphasizes short development cycles and rapid prototyping. The RAD model focuses on delivering a working system quickly by using techniques like iterative development, component-based construction, and extensive user involvement. It is best suited for projects where requirements are well-understood and the scope is manageable, allowing for faster delivery compared to traditional sequential models.

11. Comparison between prototyping and incremental process model.

Feature	Prototyping Model	Incremental Process Model
Primary Goal	Elicit and validate requirements; reduce uncertainty.	Deliver functional software in stages; early feedback.
Deliverable	A working model/mock-up (often discarded or refined).	Functional subsets of the final system.
Focus	User interface, key functionalities, user feedback.	Core functionalities, expanding features with each increment.
Nature	Exploratory, often used for requirement clarification.	Constructive, building the system piece by piece.
User Involvement	High, especially in the early stages.	High, at the end of each increment for feedback.
Risk Reduction	Reduces risk related to requirements misunderstanding.	Reduces risk by delivering working parts early.

12. Define specialized model.

Specialized process models are software development approaches tailored for specific application domains or product types, or to address particular challenges not adequately covered by generic models. These models often incorporate unique techniques, tools, or methodologies to suit the specialized context. Examples include Component-Based Development (CBD) for systems built from reusable components, Formal Methods for

safety-critical systems requiring mathematical verification, and Aspect-Oriented Software Development (AOSD) for handling cross-cutting concerns.

13. Define agile process with its methodologies.

Agile is an iterative and incremental approach to software development that emphasizes collaboration, customer feedback, and rapid delivery of working software. It prioritizes individuals and interactions over processes and tools, working software over comprehensive documentation, customer collaboration over contract negotiation, and responding to change over following a plan.

Key Agile Methodologies:

- **Scrum:** A framework for managing complex projects, emphasizing short iterations (sprints), daily stand-ups, and defined roles.
- **Kanban:** A method for visualizing work, limiting work-in-progress, and maximizing efficiency.
- **Extreme Programming (XP):** Focuses on engineering practices like pair programming, test-driven development, and continuous integration.
- **Lean Software Development:** Principles derived from lean manufacturing, focusing on eliminating waste and delivering value.

14. Write any five principles of agile.

The Agile Manifesto outlines 12 principles, from which five key principles are:

1. **Customer satisfaction** by early and continuous delivery of valuable software.
2. **Welcome changing requirements**, even late in development. Agile processes harness change for the customer's competitive advantage.
3. **Deliver working software frequently**, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. **Business people and developers must work together** daily throughout the project.
5. **Simplicity**—the art of maximizing the amount of work not done—is essential.

15. Define extreme programming.

Extreme Programming (XP) is an agile software development methodology that aims to improve software quality and responsiveness to changing customer requirements. It is characterized by short development cycles, frequent releases, and a set of core practices designed to promote teamwork, simplicity, and continuous feedback. Key practices include

pair programming, test-driven development (TDD), continuous integration, small releases, simple design, and refactoring. XP is particularly effective for projects with evolving requirements and small, co-located teams.